

# 1 Machine Learning - Criação de algoritmo de regressão e classificação.

Para a disciplina de machine learning criamos um pipeline, que inclui um algoritmo de regressão para prever o tempo de espera de um paciente em uma clínica, e um algoritmo de classificação para classificar o histórico de atraso de pacientes, se probabilidade dele se atrasar é baixa, média ou elevada.

O algoritmo de regressão utilizado foi o Random Forest Regressor e para classificação foi o Random Forest Classifier. Random Forest é um método de aprendizagem que combina múltiplas árvores de decisão para produzir respostas com acurácia alta e estáveis, usado tanto para regressão e classificação. As previsões de regressão são obtidas pela média dos resultados das diversas árvores, as previsões de classificação é a qual foi escolhida pela maioria das árvores.

## 1.1 Dicionário de dados dos arquivos usados para aprendizagem dos modelos

| Campo                                | Descrição                                       | Tipo     |
|--------------------------------------|-------------------------------------------------|----------|
| horario                              | Horário da consulta                             | Hora     |
| dia_semana                           | Dia da consulta                                 | Texto    |
| tipo_consulta                        | Especialidade da consulta                       | Texto    |
| pacientes_na_fila_antes              | Quantidade de pacientes aguardando antes        | Numérico |
| pacientes_agendados_no_mesmo_horario | Total de pacientes marcados no mesmo horário    | Numérico |
| tempo_medio_consulta_tipo_min        | Tempo médio da consulta por especialidade (min) | Numérico |
| medico_habito_atraso                 | Média de atraso do médico (min)                 | Numérico |
| horario_pico                         | Indica se o horário é de pico (sim/não)         | Texto    |
| tempo_de_espera_min                  | Tempo de espera do paciente                     | Numérico |

|                                            |                                                          |          |
|--------------------------------------------|----------------------------------------------------------|----------|
| idade                                      | Idade do paciente                                        | Numérico |
| consulta                                   | Tipo/especialidade da consulta                           | Texto    |
| distacia_local_de_partida_e_consultorio_km | Distância entre origem e consultório (km)                | Numérico |
| tipo_de_transporte                         | Meio de transporte utilizado                             | Texto    |
| horario_da_consulta                        | Horário da consulta                                      | Hora     |
| PcD                                        | Indica se o paciente é Pessoa com Deficiência            | Texto    |
| histórico_de_atrasos                       | Histórico de atrasos do paciente (baixo, médio, elevado) | Texto    |

## 1.2 Pipeline completo do algoritmo de regressão

```
#Previsão de tempo estimado de espera
df = pd.read_csv('/content/bd_tmp_espera2.csv')
df.head(5)
print(f"\nTotal de registros: {len(df)}")
print(f"\nColunas: {list(df.columns)}")
print(df.isnull().sum())
print(df.head(10))
print(df['tempo_de_espera_min'].describe())
x = df.drop('tempo_de_espera_min', axis=1)
y = df['tempo_de_espera_min'].values
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore'))
        ]), ['horario', 'dia_semana', 'tipo_consulta',
'horario_pico']),
        ('num', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='mean')),
            ('scaler', StandardScaler())
        ]), ['pacientes_na_fila_antes',
'pacientes_agendados_no_mesmo_horario',
'tempo_medio_consulta_tipo_min', 'medico_habito_atraso'])
    ],
    remainder='drop'
```

```

)
X_transform = preprocessor.fit_transform(x)
X_Train, X_Test, y_Train, y_Test = train_test_split(X_transform, y,
test_size=0.3, random_state=42)
paramgrid = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': [1.0, 'sqrt', 'log2', None],
    'bootstrap': [True, False]
}
rfr = RandomForestRegressor()
random_search = RandomizedSearchCV(
    estimator=rfr,
    param_distributions=paramgrid,
    n_iter=10,
    cv=5,
    verbose=2,
    scoring="neg_mean_squared_error",
    random_state=42,
    n_jobs=-1
)
random_search.fit(X_Train, y_Train)
best_params = random_search.best_params_
random_forest_regressor = RandomForestRegressor(**best_params)
random_forest_regressor.fit(X_Train, y_Train)
test_predic = random_forest_regressor.predict(X_Test)
mse = mean_squared_error(y_Test, test_predic)
r2 = r2_score(y_Test, test_predic)
mae = mean_absolute_error(y_Test, test_predic)
z = np.polyfit(y_Test, test_predic, 1)
p = np.poly1d(z)
plt.figure(figsize=(10, 6))
plt.scatter(y_Test, test_predic, alpha=0.6)
plt.plot(y_Test, p(y_Test), color='red', linestyle='--')
plt.xlabel('Valores Reais')
plt.ylabel('Valores Previstos')
df_to_predict = pd.read_csv('/content/bd_tmp_espera_to_predict.csv')
df_to_predict_transform = preprocessor.transform(df_to_predict)
predict = random_forest_regressor.predict(df_to_predict_transform)
df_to_predict['tempo_de_espera_min'] = predict
df_to_predict.head(5)

```

## 1.3 Pipeline completo do algoritmo de classificação

```
df_classifier = pd.read_csv('/content/data_to_train_classifier.csv')
preprocessor_classifier = ColumnTransformer(
    transformers=[
        ('cat', Pipeline(steps=[
            ('onehot', OneHotEncoder(handle_unknown='ignore'))
        ]), ['consulta', 'tipo_de_transporte', 'dia_semana',
'horario_da_consulta', 'horario_pico', 'PcD']
    ),
        ('num', Pipeline(steps=[
            ('scaler', StandardScaler())
        ]), ['idade', 'distacia_local_de_partida_e_consultorio_km'])
    ],
    remainder='drop'
)
x = df_classifier.drop('histórico_de_atrasos', axis=1)
y = df_classifier['histórico_de_atrasos'].values
label_encoder = LabelEncoder()
X_transformed = preprocessor_classifier.fit_transform(x)
Y_transformed = label_encoder.fit_transform(y)
smote = SMOTE(sampling_strategy='auto', random_state=42)
X_balanced, y_balanced = smote.fit_resample(X_transformed,
Y_transformed)
X_train, X_test, y_train, y_test = train_test_split(X_balanced,
y_balanced, test_size=0.3, random_state=42)
rfc = RandomForestClassifier()
parameters = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [10, 20, 30],
    'min_samples_leaf': [1, 2, 4, 5],
    'min_samples_split': [2, 3, 4, 5, 6, 7, 8, 9, 10],
    'max_features': [1.0, 'sqrt', 'log2', None],
    'max_samples': [0.5, 0.6, 0.7, 0.8, 0.9, 1.0],
    'bootstrap': [True, False]
}
random_searchCV = RandomizedSearchCV(
    estimator=rfc,
    param_distributions=parameters,
    n_iter=10,
    cv=5,
    verbose=2,
    scoring="neg_mean_squared_error",
```

```

    random_state=42,
    n_jobs=-1
)
random_searchCV.fit(X_train, y_train)
best_params_to_rfc = random_searchCV.best_params_
Random_Forest_Classifier = RandomForestClassifier(**best_params_to_rfc)
Random_Forest_Classifier.fit(X_train, y_train)
y_pred = Random_Forest_Classifier.predict(X_test)
conf_matrix = confusion_matrix(y_test, y_pred)
f1Score = f1_score(y_test, y_pred, average='weighted')
accuracy = accuracy_score(y_test, y_pred)
df_classifier_predict =
pd.read_csv('/content/data_to_classifier_classify.csv')
df_classifier_predict
classify_predict =
preprocessor_classifier.transform(df_classifier_predict)
predict_class = Random_Forest_Classifier.predict(classify_predict)
predictions_string = label_encoder.inverse_transform(predict_class)
df_classifier_predict['histórico_de_atrasos'] = predictions_string
df_classifier_predict.head(50)

```

## 1.4 Resultados das métricas

| Métrica                        | Resultado                                |
|--------------------------------|------------------------------------------|
| Mse (Mean Square Error)        | 2566.58                                  |
| R2 (r2 Score)                  | 0.92                                     |
| Mae (Mean Absolute Error)      | 39.80                                    |
| Conf_matrix (Confusion Matrix) | [[40 18 14]<br>[16 36 18]<br>[20 20 31]] |
| f1Score (f1 score)             | 0.50                                     |
| accuracy (accuracy score)      | 0.50                                     |

## Referências

GEEKSFORGEEKS. Random Forest Regression in Python. *GeeksforGeeks*, 14 jun. 2019. Atualizado em: 6 abr. 2026. Disponível em: <https://www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/>. Acesso em: 19 maio 2026.

GEEKSFORGEEKS. Random Forest Classifier using Scikit-learn. *GeeksforGeeks*, 4 set. 2020. Atualizado em: 6 mar. 2026. Disponível em: <https://www.geeksforgeeks.org/random-forest-classifier-using-scikit-learn/>. Acesso em: 19 maio 2026.